



**AWD COURSE**

Platform Pembelajaran Pemrograman & Analisis Data

MODUL REFERENSI RESMI • BAB 3

# Analisis Data Eksploratif (EDA) dengan Python

Panduan komprehensif inspeksi struktur data (head, tail, info), kalkulasi statistik deskriptif mean/median/std, visualisasi distribusi histogram dan scatter plot, serta teknik seleksi data dan boolean indexing.

KATEGORI MATERI

**Dasar Python & Analisis Data**

FORMAT DOKUMEN

**PDF Interaktif Siap Cetak**

TINGKAT KESULITAN

**Pemula s/d Menengah**

AKSES ONLINE

**`course.awd.my.id`**

PEMATERI & PENYUSUN MATERI

**I Putu Agus Wahyu Dupayana**

Dipublikasikan secara resmi di AWD Course Platform

Edisi Pembaruan: **September 2026**

## Bab 3: Analisis Data Eksploratif (EDA)

---

Awd Course – Buku Modul Praktik Komprehensif

Notebook kompilasi lengkap untuk Bab 3 yang mencakup Inspeksi Struktur Dataframe, Statistik Deskriptif, Visualisasi Distribusi, dan Seleksi Data Boolean.

---

### Persiapan & Inspeksi Awal Struktur Dataframe

---

**Analisis Data Eksploratif (Exploratory Data Analysis – EDA)** adalah tahapan kritis pertama dalam sains data untuk memahami bentuk fisik data, tipe data setiap variabel, anomali, dan kualitas data sebelum melakukan pemodelan statistik.

## 1. Menyiapkan & Memuat Dataset Wilayah

PYTHON 3

Contoh Kode Praktik

```
import pandas as pd
import numpy as np

# Menyiapkan dataset demografi wilayah (440 baris) yang self-contained & representatif
np.random.seed(42)
n_samples = 440

nama_wilayah_list = [
    'Los_Angeles', 'Cook', 'Harris', 'San_Diego', 'Orange', 'Maricopa', 'Dallas',
    'Miami_Dade', 'King', 'Clark', 'Wayne', 'Tarrant', 'Bexar', 'Santa_Clara',
    'Wayne', 'Alameda', 'Middlesex', 'Hennepin', 'Sacramento', 'Travis', 'Fulton'
]
klasifikasi_list = ['West', 'Midwest', 'South', 'Northeast']

df = pd.DataFrame({
    'Nomor Identitas Wilayah': np.arange(1, n_samples + 1),
    'Nama Wilayah': [nama_wilayah_list[i % len(nama_wilayah_list)] for i in range(n_samples)],
    'Kode Wilayah': ['CA', 'IL', 'TX', 'CA', 'CA', 'AZ', 'TX', 'FL', 'WA', 'NV'][0:n_samples] if
n_samples <= 10 else ['CA', 'IL', 'TX', 'CA', 'CA', 'AZ', 'TX', 'FL', 'WA', 'NV'] * (n_samples // 10)
+ ['CA'] * (n_samples % 10),
    'Luas Wilayah (mil2)': np.random.randint(15, 4500, size=n_samples),
    'Jumlah Penduduk (jiwa)': np.random.randint(100000, 3000000, size=n_samples),
    'Persen Penduduk 18-34th': np.round(np.random.uniform(16.0, 45.0, size=n_samples), 2),
    'Persen Penduduk >64th': np.round(np.random.uniform(5.0, 25.0, size=n_samples), 2),
    'Jumlah Dokter': np.random.randint(50, 15000, size=n_samples),
    'Jumlah Kejadian Kriminal': np.random.randint(500, 500000, size=n_samples),
    'Persen Lulusan Sarjana/Diploma IV': np.round(np.random.uniform(10.0, 50.0, size=n_samples), 2),
    'Persen Pengangguran': np.round(np.random.uniform(2.5, 15.0, size=n_samples), 2),
    'Pendapatan Perkapita': np.random.randint(9000, 38000, size=n_samples),
    'Klasifikasi Wilayah': np.random.choice(klasifikasi_list, size=n_samples, p=[0.35, 0.25, 0.25,
0.15])
})

print(f"Dataset berhasil dimuat: {df.shape[0]} baris x {df.shape[1]} kolom")
```

OUTPUT EKSEKUSI:

Dataset berhasil dimuat: 440 baris x 13 kolom

## 2. Melihat Baris Teratas ( `df.head()` ) dan Terbawah ( `df.tail()` )

- `df.head(n)` : Menampilkan \$n\$ baris pertama untuk memverifikasi kesesuaian penamaan kolom dan contoh nilai.
- `df.tail(n)` : Menampilkan \$n\$ baris terakhir.

```
# Melihat 5 baris pertama
print("5 Baris Pertama:")
df.head()
```

**OUTPUT EKSEKUSI:**

5 Baris Pertama:

**OUTPUT EKSEKUSI:**

| Nomor | Identitas Wilayah | Nama Wilayah | Kode Wilayah | Luas Wilayah (mil2) | \ |
|-------|-------------------|--------------|--------------|---------------------|---|
| 0     | 1                 | Los_Angeles  | CA           | 875                 |   |
| 1     | 2                 | Cook         | IL           | 3787                |   |
| 2     | 3                 | Harris       | TX           | 3107                |   |
| 3     | 4                 | San_Diego    | CA           | 481                 |   |
| 4     | 5                 | Orange       | CA           | 4441                |   |

|   | Jumlah Penduduk (jiwa) | Persen Penduduk 18-34th | Persen Penduduk >64th | \ |
|---|------------------------|-------------------------|-----------------------|---|
| 0 | 1139458                | 39.19                   | 8.86                  |   |
| 1 | 2417336                | 39.93                   | 6.68                  |   |
| 2 | 2863642                | 21.41                   | 11.53                 |   |
| 3 | 1909891                | 22.84                   | 16.20                 |   |
| 4 | 2519862                | 34.38                   | 7.32                  |   |

|   | Jumlah Dokter | Jumlah Kejadian Kriminal | Persen Lulusan Sarjana/Diploma IV | \ |
|---|---------------|--------------------------|-----------------------------------|---|
| 0 | 4305          | 339615                   | 32.41                             |   |
| 1 | 14534         | 356115                   | 40.03                             |   |
| 2 | 11065         | 327590                   | 32.32                             |   |
| 3 | 7447          | 361633                   | 32.10                             |   |
| 4 | 1785          | 84580                    | 13.77                             |   |

|   | Persen Pengangguran | Pendapatan Perkapita | Klasifikasi Wilayah |
|---|---------------------|----------------------|---------------------|
| 0 | 14.12               | 12664                | Midwest             |
| 1 | 2.88                | 28873                | Midwest             |
| 2 | 13.40               | 22050                | South               |
| 3 | 9.09                | 9680                 | Midwest             |
| 4 | 10.41               | 27908                | Midwest             |

### 3. Memeriksa Metadata & Tipe Data Kolom ( `df.info()` )

Metode `df.info()` memberikan ringkasan esensial: 1. Total entri baris (*RangeIndex*). 2. Jumlah kolom beserta nama kolomnya. 3. Nilai yang terisi (*Non-Null Count*) untuk mendeteksi *missing values*. 4. Tipe data masing-masing kolom (*int64*, *float64*, atau *object/string*). 5. Penggunaan memori (*memory usage*).

```
# Memeriksa informasi struktur dataframe
df.info()
```

#### OUTPUT EKSEKUSI:

```
<class 'pandas.DataFrame'>
RangeIndex: 440 entries, 0 to 439
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Nomor Identitas Wilayah              440 non-null    int64
1   Nama Wilayah                        440 non-null    str
2   Kode Wilayah                        440 non-null    str
3   Luas Wilayah (mil2)                 440 non-null    int32
4   Jumlah Penduduk (jiwa)              440 non-null    int32
5   Persen Penduduk 18-34th             440 non-null    float64
6   Persen Penduduk >64th               440 non-null    float64
7   Jumlah Dokter                       440 non-null    int32
8   Jumlah Kejadian Kriminal            440 non-null    int32
9   Persen Lulusan Sarjana/Diploma IV   440 non-null    float64
10  Persen Pengangguran                 440 non-null    float64
11  Pendapatan Perkapita                440 non-null    int32
12  Klasifikasi Wilayah                 440 non-null    str
dtypes: float64(4), int32(5), int64(1), str(3)
memory usage: 36.2 KB
```

## 4. Memeriksa Dimensi dan Nama Kolom

```
print("Dimensi Dataframe (Baris, Kolom):", df.shape)
print("\nDaftar Nama Kolom:\n", df.columns.tolist())
```

#### OUTPUT EKSEKUSI:

```
Dimensi Dataframe (Baris, Kolom): (440, 13)
```

```
Daftar Nama Kolom:
```

```
['Nomor Identitas Wilayah', 'Nama Wilayah', 'Kode Wilayah', 'Luas Wilayah (mil2)', 'Jumlah Penduduk (jiwa)',
'Persen Penduduk 18-34th', 'Persen Penduduk >64th', 'Jumlah Dokter', 'Jumlah Kejadian Kriminal', 'Persen
Lulusan Sarjana/Diploma IV', 'Persen Pengangguran', 'Pendapatan Perkapita', 'Klasifikasi Wilayah']
```

## Statistik Deskriptif & Visualisasi Plot EDA

Setelah memahami struktur awal dataset, langkah selanjutnya adalah mengekstrak **ringkasan statistik kuantitatif** (ukuran pemusatan dan penyebaran data) serta visualisasi distribusinya.

## 1. Menyiapkan Dataset

PYTHON 3

Contoh Kode Praktik

```
import pandas as pd
import numpy as np

# Menyiapkan dataset demografi wilayah (440 baris) yang self-contained & representatif
np.random.seed(42)
n_samples = 440

nama_wilayah_list = [
    'Los_Angeles', 'Cook', 'Harris', 'San_Diego', 'Orange', 'Maricopa', 'Dallas',
    'Miami_Dade', 'King', 'Clark', 'Wayne', 'Tarrant', 'Bexar', 'Santa_Clara',
    'Wayne', 'Alameda', 'Middlesex', 'Hennepin', 'Sacramento', 'Travis', 'Fulton'
]
klasifikasi_list = ['West', 'Midwest', 'South', 'Northeast']

df = pd.DataFrame({
    'Nomor Identitas Wilayah': np.arange(1, n_samples + 1),
    'Nama Wilayah': [nama_wilayah_list[i % len(nama_wilayah_list)] for i in range(n_samples)],
    'Kode Wilayah': ['CA', 'IL', 'TX', 'CA', 'CA', 'AZ', 'TX', 'FL', 'WA', 'NV'][0:n_samples] if
n_samples <= 10 else ['CA', 'IL', 'TX', 'CA', 'CA', 'AZ', 'TX', 'FL', 'WA', 'NV'] * (n_samples // 10)
+ ['CA'] * (n_samples % 10),
    'Luas Wilayah (mil2)': np.random.randint(15, 4500, size=n_samples),
    'Jumlah Penduduk (jiwa)': np.random.randint(100000, 3000000, size=n_samples),
    'Persen Penduduk 18-34th': np.round(np.random.uniform(16.0, 45.0, size=n_samples), 2),
    'Persen Penduduk >64th': np.round(np.random.uniform(5.0, 25.0, size=n_samples), 2),
    'Jumlah Dokter': np.random.randint(50, 15000, size=n_samples),
    'Jumlah Kejadian Kriminal': np.random.randint(500, 500000, size=n_samples),
    'Persen Lulusan Sarjana/Diploma IV': np.round(np.random.uniform(10.0, 50.0, size=n_samples), 2),
    'Persen Pengangguran': np.round(np.random.uniform(2.5, 15.0, size=n_samples), 2),
    'Pendapatan Perkapita': np.random.randint(9000, 38000, size=n_samples),
    'Klasifikasi Wilayah': np.random.choice(klasifikasi_list, size=n_samples, p=[0.35, 0.25, 0.25,
0.15])
})

print(f"Dataset berhasil dimuat: {df.shape[0]} baris x {df.shape[1]} kolom")
```

OUTPUT EKSEKUSI:

Dataset berhasil dimuat: 440 baris x 13 kolom

## 2. Ringkasan Statistik 5 Angka ( `df.describe()` )

Fungsi `df.describe()` menghitung secara otomatis:

- `count` : Jumlah data yang tidak bernilai null.
- `mean` : Nilai rata-rata aritmatika.
- `std` : Standar deviasi (penyebaran data dari rata-rata).
- `min` : Nilai minimum.

- **25%** (Kuartil 1 / Q1), **50%** (Median / Kuartil 2), **75%** (Kuartil 3 / Q3).
- **max** : Nilai maksimum.

#### PYTHON 3

Contoh Kode Praktik

```
# Menampilkan ringkasan statistik deskriptif untuk seluruh kolom numerik
df.describe()
```

#### OUTPUT EKSEKUSI:

| Nomor | Identitas Wilayah | Luas Wilayah (mil2) | Jumlah Penduduk (jiwa) | \ |
|-------|-------------------|---------------------|------------------------|---|
| count | 440.000000        | 440.000000          | 4.400000e+02           |   |
| mean  | 220.500000        | 2276.765909         | 1.536295e+06           |   |
| std   | 127.161315        | 1243.820185         | 8.292159e+05           |   |
| min   | 1.000000          | 19.000000           | 1.004040e+05           |   |
| 25%   | 110.750000        | 1162.750000         | 8.414990e+05           |   |
| 50%   | 220.500000        | 2391.500000         | 1.472964e+06           |   |
| 75%   | 330.250000        | 3283.250000         | 2.310423e+06           |   |
| max   | 440.000000        | 4489.000000         | 2.998073e+06           |   |

|       | Persen Penduduk 18-34th | Persen Penduduk >64th | Jumlah Dokter | \ |
|-------|-------------------------|-----------------------|---------------|---|
| count | 440.000000              | 440.000000            | 440.000000    |   |
| mean  | 30.983818               | 15.044864             | 7941.497727   |   |
| std   | 8.300521                | 5.557366              | 4148.108149   |   |
| min   | 16.010000               | 5.020000              | 66.000000     |   |
| 25%   | 23.915000               | 10.110000             | 4631.250000   |   |
| 50%   | 31.285000               | 15.495000             | 8023.000000   |   |
| 75%   | 37.995000               | 19.645000             | 11327.000000  |   |
| max   | 44.980000               | 24.870000             | 14966.000000  |   |

|       | Jumlah Kejadian Kriminal | Persen Lulusan Sarjana/Diploma IV | \ |
|-------|--------------------------|-----------------------------------|---|
| count | 440.000000               | 440.000000                        |   |
| mean  | 234216.295455            | 29.835750                         |   |
| std   | 143461.894459            | 11.633221                         |   |
| min   | 1281.000000              | 10.010000                         |   |
| 25%   | 111385.000000            | 19.852500                         |   |
| 50%   | 230685.500000            | 29.520000                         |   |
| 75%   | 355441.500000            | 39.870000                         |   |
| max   | 499185.000000            | 49.990000                         |   |

|       | Persen Pengangguran | Pendapatan Perkapita |
|-------|---------------------|----------------------|
| count | 440.000000          | 440.000000           |
| mean  | 8.991227            | 23316.952273         |
| std   | 3.645562            | 8516.301587          |
| min   | 2.510000            | 9060.000000          |
| 25%   | 5.837500            | 15393.500000         |
| 50%   | 9.320000            | 23634.500000         |
| 75%   | 12.162500           | 30553.000000         |
| max   | 14.990000           | 37985.000000         |

### 3. Visualisasi Boxplot Sebaran per Wilayah

**Boxplot** sangat berguna untuk mendeteksi:

- Median garis tengah.
- Rentang interkuartil ( $IQR = Q3 - Q1$ ).
- Titik-titik pencilan (*outliers*).

PYTHON 3

Contoh Kode Praktik

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
df.boxplot(column='Pendapatan Perkapita', by='Klasifikasi Wilayah', grid=True)
plt.title('Perbandingan Distribusi Pendapatan Perkapita per Wilayah')
plt.suptitle('') # Hapus default subtitle bawaan pandas
plt.xlabel('Klasifikasi Wilayah')
plt.ylabel('Pendapatan Perkapita (USD)')
plt.show()
```

OUTPUT EKSEKUSI:

<Figure size 1000x600 with 0 Axes>

OUTPUT EKSEKUSI:

<Figure size 640x480 with 1 Axes>

## 4. Scatter Plot: Korelasi Tingkat Pendidikan vs Pendapatan

PYTHON 3

Contoh Kode Praktik

```
plt.figure(figsize=(9, 5))
df.plot.scatter(
    x='Persen Lulusan Sarjana/Diploma IV',
    y='Pendapatan Perkapita',
    title='Hubungan Antara Tingkat Pendidikan Sarjana & Pendapatan Perkapita',
    c='Persen Pengangguran',
    colormap='viridis',
    figsize=(10, 6),
    sharex=False
)
plt.show()
```

OUTPUT EKSEKUSI:

<Figure size 900x500 with 0 Axes>

OUTPUT EKSEKUSI:

<Figure size 1000x600 with 2 Axes>



# Seleksi Kolom & Filtering Data Boolean

Kemampuan memilih (*selecting*) kolom dan menyaring (*filtering*) baris data berdasarkan kriteria tertentu adalah keterampilan paling esensial dalam analisis data sehari-hari.

## 1. Menyiapkan Dataset

PYTHON 3

Contoh Kode Praktik

```
import pandas as pd
import numpy as np

# Menyiapkan dataset demografi wilayah (440 baris) yang self-contained & representatif
np.random.seed(42)
n_samples = 440

nama_wilayah_list = [
    'Los_Angeles', 'Cook', 'Harris', 'San_Diego', 'Orange', 'Maricopa', 'Dallas',
    'Miami_Dade', 'King', 'Clark', 'Wayne', 'Tarrant', 'Bexar', 'Santa_Clara',
    'Wayne', 'Alameda', 'Middlesex', 'Hennepin', 'Sacramento', 'Travis', 'Fulton'
]
klasifikasi_list = ['West', 'Midwest', 'South', 'Northeast']

df = pd.DataFrame({
    'Nomor Identitas Wilayah': np.arange(1, n_samples + 1),
    'Nama Wilayah': [nama_wilayah_list[i % len(nama_wilayah_list)] for i in range(n_samples)],
    'Kode Wilayah': ['CA', 'IL', 'TX', 'CA', 'CA', 'AZ', 'TX', 'FL', 'WA', 'NV'][0:n_samples] if
n_samples <= 10 else ['CA', 'IL', 'TX', 'CA', 'CA', 'AZ', 'TX', 'FL', 'WA', 'NV'] * (n_samples // 10)
+ ['CA'] * (n_samples % 10),
    'Luas Wilayah (mil2)': np.random.randint(15, 4500, size=n_samples),
    'Jumlah Penduduk (jiwa)': np.random.randint(100000, 3000000, size=n_samples),
    'Persen Penduduk 18-34th': np.round(np.random.uniform(16.0, 45.0, size=n_samples), 2),
    'Persen Penduduk >64th': np.round(np.random.uniform(5.0, 25.0, size=n_samples), 2),
    'Jumlah Dokter': np.random.randint(50, 15000, size=n_samples),
    'Jumlah Kejadian Kriminal': np.random.randint(500, 500000, size=n_samples),
    'Persen Lulusan Sarjana/Diploma IV': np.round(np.random.uniform(10.0, 50.0, size=n_samples), 2),
    'Persen Pengangguran': np.round(np.random.uniform(2.5, 15.0, size=n_samples), 2),
    'Pendapatan Perkapita': np.random.randint(9000, 38000, size=n_samples),
    'Klasifikasi Wilayah': np.random.choice(klasifikasi_list, size=n_samples, p=[0.35, 0.25, 0.25,
0.15])
})

print(f"Dataset berhasil dimuat: {df.shape[0]} baris x {df.shape[1]} kolom")
```

### OUTPUT EKSEKUSI:

Dataset berhasil dimuat: 440 baris x 13 kolom

## 2. Seleksi Kolom (Series vs DataFrame)

- `df['Nama Kolom']` : Mengambil 1 kolom sebagai **Series**.
- `df[['Kolom1', 'Kolom2']]` : Mengambil beberapa kolom sekaligus sebagai **DataFrame**.

PYTHON 3

Contoh Kode Praktik

```
# 1. Mengambil satu kolom (Series)
wilayah_series = df['Nama Wilayah']
print("Tipe 1 Kolom:", type(wilayah_series))
print(wilayah_series.head(3))

# 2. Mengambil subset beberapa kolom (DataFrame)
subset_df = df[['Nama Wilayah', 'Jumlah Penduduk (jiwa)', 'Pendapatan Perkapita']]
print("\nTipe Banyak Kolom:", type(subset_df))
subset_df.head(3)
```

### OUTPUT EKSEKUSI:

```
Tipe 1 Kolom: <class 'pandas.Series'>
0    Los_Angeles
1         Cook
2         Harris
Name: Nama Wilayah, dtype: str
```

```
Tipe Banyak Kolom: <class 'pandas.DataFrame'>
```

### OUTPUT EKSEKUSI:

|   | Nama Wilayah | Jumlah Penduduk (jiwa) | Pendapatan Perkapita |
|---|--------------|------------------------|----------------------|
| 0 | Los_Angeles  | 1139458                | 12664                |
| 1 | Cook         | 2417336                | 28873                |
| 2 | Harris       | 2863642                | 22050                |

## 3. Filter Baris Menggunakan Boolean Indexing

Kita membuat kondisi logika Boolean yang menghasilkan `True` atau `False` pada setiap baris.

```
# Filter 1: Wilayah dengan Penduduk > 1.5 Juta Jiwa
populasi_tinggi = df[df['Jumlah Penduduk (jiwa)'] > 1500000]
print(f"Jumlah wilayah berpenduduk > 1.5 juta: {len(populasi_tinggi)}")
populasi_tinggi[['Nama Wilayah', 'Jumlah Penduduk (jiwa)', 'Klasifikasi Wilayah']].head()
```

**OUTPUT EKSEKUSI:**

Jumlah wilayah berpenduduk > 1.5 juta: 215

**OUTPUT EKSEKUSI:**

|   | Nama Wilayah | Jumlah Penduduk (jiwa) | Klasifikasi Wilayah |
|---|--------------|------------------------|---------------------|
| 1 | Cook         | 2417336                | Midwest             |
| 2 | Harris       | 2863642                | South               |
| 3 | San_Diego    | 1909891                | Midwest             |
| 4 | Orange       | 2519862                | Midwest             |
| 6 | Dallas       | 2486488                | South               |

## 4. Filter dengan Multi Kondisi (& untuk AND, | untuk OR)

 **Catatan Penting:** Di Pandas, setiap kondisi logika harus diapit tanda kurung (kondisi1) & (kondisi2) .

```
# Filter: Wilayah 'West' DENGAN Pendapatan Perkapita > 25.000
filter_multi = df[(df['Klasifikasi Wilayah'] == 'West') & (df['Pendapatan Perkapita'] > 25000)]
print(f"Ditemukan {len(filter_multi)} wilayah yang memenuhi kriteria:")
filter_multi[['Nama Wilayah', 'Kode Wilayah', 'Pendapatan Perkapita', 'Klasifikasi Wilayah']].head()
```

**OUTPUT EKSEKUSI:**

Ditemukan 51 wilayah yang memenuhi kriteria:

**OUTPUT EKSEKUSI:**

|    | Nama Wilayah | Kode Wilayah | Pendapatan Perkapita | Klasifikasi Wilayah |
|----|--------------|--------------|----------------------|---------------------|
| 16 | Middlesex    | TX           | 27287                | West                |
| 21 | Los_Angeles  | IL           | 29544                | West                |
| 36 | Alameda      | TX           | 27919                | West                |
| 39 | Sacramento   | NV           | 35400                | West                |
| 47 | Maricopa     | FL           | 28356                | West                |

## 5. Metode Praktis: `isin()` dan `query()`

- `isin([ ... ])` : Memfilter baris yang nilainya cocok dengan salah satu anggota daftar.
- `query(" ... ")` : Menyaring data dengan sintaksis ekspresi string yang sangat bersih.

```
# Menggunakan isin()
df_west_midwest = df[df['Klasifikasi Wilayah'].isin(['West', 'Midwest'])]
print("Jumlah wilayah West & Midwest:", len(df_west_midwest))

# Menggunakan query()
df_hasil_query = df.query("`Pendapatan Perkapita` > 30000 and `Persen Pengangguran` < 5.0")
print("Wilayah pendapatan tinggi dengan pengangguran rendah:", len(df_hasil_query))
df_hasil_query[['Nama Wilayah', 'Pendapatan Perkapita', 'Persen Pengangguran']]
```

**OUTPUT EKSEKUSI:**

Jumlah wilayah West & Midwest: 249

Wilayah pendapatan tinggi dengan pengangguran rendah: 23

**OUTPUT EKSEKUSI:**

|     | Nama Wilayah | Pendapatan Perkapita | Persen Pengangguran |
|-----|--------------|----------------------|---------------------|
| 15  | Alameda      | 34675                | 2.53                |
| 45  | San_Diego    | 32255                | 4.31                |
| 76  | Santa_Clara  | 36798                | 4.00                |
| 87  | San_Diego    | 33335                | 4.86                |
| 120 | Alameda      | 31724                | 4.12                |
| 135 | Clark        | 31742                | 2.54                |
| 137 | Tarrant      | 33703                | 4.83                |
| 150 | San_Diego    | 31077                | 4.07                |
| 153 | Dallas       | 31544                | 3.15                |
| 157 | Wayne        | 34917                | 3.57                |
| 163 | Middlesex    | 31192                | 3.75                |
| 177 | Clark        | 33729                | 3.02                |
| 213 | San_Diego    | 34135                | 3.25                |
| 240 | Clark        | 33351                | 3.26                |
| 257 | Maricopa     | 33648                | 4.75                |
| 264 | Bexar        | 34640                | 4.21                |
| 278 | Maricopa     | 31520                | 4.70                |
| 372 | Alameda      | 32936                | 3.10                |
| 394 | Middlesex    | 30225                | 3.15                |
| 406 | Miami_Dade   | 34142                | 4.07                |
| 413 | Wayne        | 37985                | 4.37                |
| 417 | Sacramento   | 34812                | 2.76                |
| 438 | Sacramento   | 36080                | 4.06                |